

CENTRO NACIONAL DE PESQUISA EM ENERGIAS E MATERIAIS
|
CNPEM
ILUM - ESCOLA DE CIÊNCIA

RELATÓRIO DE PRÁTICA EM CIÊNCIA DE DADOS
Projeto final: Simulação de Gradiente de um Neurônio

ALUNOS:

VINÍCIUS MARIANNO DE MARQUE CUTOLO

PROFESSOR:

DANIEL ROBERTO CASSAR

JAMES MORAES DE ALMEIDA

LEANDRO NASCIMENTO LEMOS

13 de Junho de 2026
CAMPINAS – SP | BRASIL

Resumo

O projeto a seguir busca simular neurônios isolados e em rede e tentar prever a composição de um sinal, dado um fragmento dele. Nesse quesito, ele obteve um bom desempenho em todas as tarefas, representando um desafio ao aluno e um produto interessante.

Palavras-chave: Neurônio, Simulação, Previsão

Agradecimento

Gostaria de dedicar essa sessão a duas pessoas que foram fundamentais para a construção desse projeto. A primeira foi o professor Leandro Nascimento Lemos, que disponibilizou todo o material que fundamentou, tanto no ramo da teoria quanto no ramo prático, tudo que foi criado. Ademais, queria agradecer o meu colega Filipi Martins, que ajudou com ideias e conteúdos para iniciar o estudo de alguns tópicos mais avançados dessa simulação.

Introdução

Esse projeto surgiu como uma tentativa de simular o potencial de membrana de um neurônio e expressar isso em uma interface gráfica. Contudo, outros objetivos também foram buscados, como conectar os neurônios em um grafo para simular um eletroencefalograma (EEG) ou pegar um sinal de EEG e tentar prever a sua composição em termos de frações de sinais.

O modelo usado para testar o neurônio foi o modelo de Hodgkin-Huxley Markoviano [1] [2] com algumas adições como canais não lineares de cálcio, correntes sinápticas e função noise, com o método de atualização de Runge Kutta de 4º ordem [3]. Isso permitia criar um neurônio com uma resposta interessante e adaptativo, apresentando memória no curto prazo. A simulação de EEG não apresentou nenhuma mecânica adicional, somente conectando os neurônios dentro de uma rede.

Ademais, o algoritmo de previsão de sinal usou Fast Fourier Transform (FFT) e Orthogonal Matching Pursuit (OMP) para calcular e prever a composição de sinais, resultando em um algoritmo estimativo capaz de prever composição de sinais com componentes altas de composição.

Por fim, a simulação tem como principal objetivo iniciar os estudos no ramo da computação neurológica e neuromórfica. Mesmo que ele não possa ser 100% representativo da realidade, ainda há resultados interessantes vistos dentro dela, capazes de explicarem algumas características observáveis.

Metodologia

Métodos

Para simular as características dos neurônios, algumas coisas foram feitas seguindo rigidamente a literatura e outras não. Para simular os canais iônicos, as funções foram todas usadas de

acordo com a referência [2], seguindo o modelo que será comentado posteriormente. Contudo, algumas variáveis necessitavam ser adaptadas para o correto funcionamento do programa, o valor de literatura para o equilíbrio entre a abertura e fechamento dos canais levava o valor real, que depende de muitas outras variáveis não incertas. Por isso, foi observado a forma do potencial de membrana e, a fim de se aproximar dela, as constantes de equilíbrio foram calibradas.

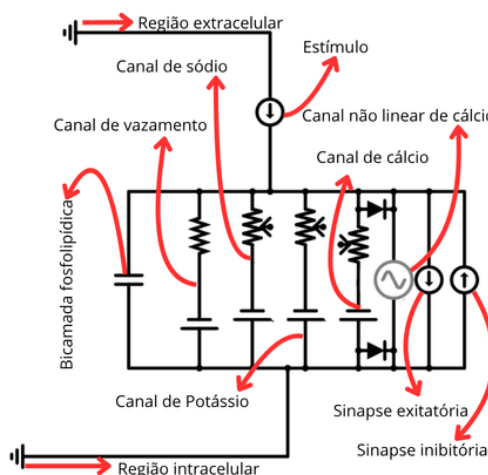
Por isso, não é possível confiar no modelo proposto para uma análise tão profunda, já que, nele, usa-se números artificiais, se tornando uma simulação mais qualitativa do que um modelo robusto por si.

Resultados e discussão

0.1 Arcabouço teórico

Essa seção será dedicada para introduzir o leitor no contexto necessário para compreender o código do projeto.

O modelo Hodgkin-Huxley é uma forma de aproximar o neurônio para um circuito elétrico. Para isso, dividimos cada componente em uma subunidade: a bicamada fosfolipídica se torna um capacitor, os canais iônicos são reostatos (resistores de resistência variável) em série com uma fonte de tensão, as sinapses são fontes de corrente, o acúmulo de cálcio é representado como uma fonte de tensão controlada por corrente (FTCC) e os estímulos são fontes de correntes. Todos esses elementos são organizados como na imagem a seguir 1.



(a) 1

Figura 1: Circuito neurônio

Fonte: elaborado pelo criador, usando Falstad.

Com isso, temos um modelo base que permite calcular o potencial de membrana, basta usar a equação do capacitor (1) e imaginar que toda carga acumulada nele virá do somatório das correntes (2).

$$V = \frac{Q}{C} \quad (1)$$

$$dQ_t = (I_{estimulo} - \sum_{i=1}^n I_i)dt \quad (2)$$

Isso nos dá uma equação diferencial para o potencial de membrana. Contudo, não é suficiente, nos canais iônicos lineares há a presença de reostatos, para calcular sua resistência é usado o conceito de Cadeia de Markov, uma ferramenta matemática que permite com que o estado passado afete o futuro próximo (efeito de memória). A única exceção a isso é o canal de vazamento, que tem uma variação ínfima, então foi considerado como um resistor fixo.

Para calcular as resistências, considerou-se que cada reostato, no contexto de cada canal do neurônio, só pode estar em dois estados: aberto ou fechado. Assim, não é necessário tratar cada um deles, e sim qual a porcentagem deles que está aberta. Para isso, um vetor de estado foi definido para saber qual o percentil de cada canal em cada estado, usando matrizes de transição para evoluir os a depender da diferença de potencial (d.d.p) da membrana.

Por fim, os canais de cálcio são separados em dois: um que segue o esquema linear e o outro que representa o acúmulo interno de cálcio no neurônio. Esse segundo tem sua corrente definida pela corrente do canal de cálcio, calculando o acúmulo do íon dentro da célula gerando um efeito repelidor para correntes entrando.

Para atualizar esses valores, um método de aproximação chamado RK4 foi utilizado. O método de Euler trata aproximações para integrais como multiplicações discretas em uma equação do tipo $\Delta X = \sum_{i=0}^n f(x_i) \frac{x_{i+1} - x_i}{k}$, quanto maior é k , menor o erro da aproximação. Contudo, para um uso contínuo, essa fórmula acumula muito erro.

Para mitigar isso, o método RK4 faz o uso de uma ideia parecida. Ele ainda usa dx em sua forma discreta, mas ele faz isso interpolando em quatro pontos de uma curva. Se a função derivada existe, pega-se o ponto inicial $f(x, y)$ e encontra-se a sua inclinação, essa inclinação se chama k_1 , o processo é repetido para outros três pontos achando as seguintes inclinações $f(x + \frac{dx}{2}, y + k_1 \frac{dx}{2})$, $f(x + \frac{dx}{2}, y + k_2 \frac{dx}{2})$ e $f(x + dx, y + k_3 dx)$.

Com todas as inclinações adquiridas, soma-se elas em uma média ponderada, diminuindo o erro dos passos discretos.

Por fim, algumas técnicas utilizadas em **FFT_functions**. A parte mais essencial dela está na **OMP** [4], isso é uma ferramenta usada para calcular sistemas lineares que possuem matrizes esparsas. De uma forma básica, ele analisa o resíduo de um sinal e busca, entre as ondas possíveis, qual o melhor resultado que explica esse resíduo. Isso é iterado n vezes até sua solução atingir um score ou erro desejado.

Contudo, como os sinais são analisados em fragmentos, o número de combinações rapidamente explode para um número exorbitante ($(\Delta p)^n$ combinações, com p sendo $p = d - t + 1$, com d o tamanho do sinal e t o tamanho do fragmento, e n o número de sinais distintos). Para isso, usou-se uma técnica chamada Coarse-to-Fine.

Essa técnica se baseia na ideia que soluções ótimas são rodeadas por uma vizinhança de score alto e erro baixo. Tal ideia fundamenta um método de busca, nele, a busca é feita com um passo alto antes, ranqueando os resultados e usando os k melhores para retroalimentar na próxima iteração. Isso diminui o número de buscas enormemente, aumentando muito a performance.

do programa. Contudo, existe o sacrifício da possível melhor solução, já que nem todas as combinações serão testadas. Normalmente, é uma troca boa, já que o resultado não é tão pior que seria se rodasse todas as combinações.

0.2 Código do menu

Para analisar e explicar esse código, cada função relevante definida será colocada e comentada sobre sua utilidade.

```

1  from PIL import Image
2  import keyboard as kb
3  import subprocess
4  import pygame
5  import json
6  import math
7  import sys
8  import os
9
10  caminho = os.path.join(os.path.dirname(__file__), "Simulations")
11  sys.path.append(caminho)
12
13  # Eu estou ignorando porque esse erro e resolvido pela parte de
14  cima
15  import FFT_functions as fft # type: ignore
16
17  # Faz o pygame aparecer na esquerda
18  os.environ["SDL_VIDEO_WINDOW_POS"] = "240,100"

```

Esse trecho importa todas as bibliotecas necessárias, cada uma será melhor discutida quando aparecer ao longo do código, portanto, aqui só será dita uma descrição básica.

Image do **PIL** é usada para abrir e manipular outros formatos de imagem (nesse contexto, um gif). **Keyboard** serve para pegar verificar facilmente se uma tecla está sendo pressionada. **Subprocess** permite abrir outros scripts fora do ambiente principal, não afetando o motor gráfico principal. **Pygame** é uma biblioteca usada para gerar uma interface gráfica parecida com um jogo. **Json** serve para manipular arquivos .json. **Math** permite usar algumas funções matemáticas prontas. **Sys** permite acessar informações do compilador python. Por último, **os** permite interagir com o sistema operacional.

O import **FFT_functions** é um script próprio, e será discutido em sua própria sessão. A linha entre esses imports permite importar a **FFT_functions** como se ele estivesse na mesma pasta que o script do menu, basicamente ele pega o caminho da pasta **Simulations** e adiciona nos caminhos de busca de biblioteca do compilador python.

Por fim, a última linha define onde que a janela do **pygame** irá abrir.

O código prosegue para definir algumas funções, a maioria delas, como **justificar_texto** ou **componentes_reais** são funções estética de formatação de texto. Servem para automatizar o processo de quebra de linha ou de geração de strings para resultados. As exceções são a função **is_in** e a **carregar_gif**.

A primeira é muito simples, ela retorna direto uma operação do **pygame** que calcula a colisão da posição com um dado sprite, já a segunda é um pouco mais complexa. O **Image** permite carregar formatos de imagem não usuais, ao carregar um gif em uma variável ela inicia já no primeiro frame do gif. Para ser possível utilizar o modo animado do gif, a função itera sobre cada frame, convertendo ele em uma bitstring em RGBA e criando um objeto de imagem do **pygame**. Cada frame é armazenado em uma lista que será, posteriormente, iterada para fazer o fundo animado.

```

1  def carregar_gif(caminho: str):
2      """Decompoe um gif em frames e guarda eles em uma lista"""
3      gif = Image.open(caminho)
4
5      frames = []
6      try:
7          # Colocando cada frame dentro de frames
8          while True:
9              frame = gif.convert("RGBA")
10             frame = pygame.image.fromstring(frame.tobytes(),
11                 frame.size, "RGBA")
12             frames.append(frame)
13             gif.seek(gif.tell() + 1)
14         except EOFError:
15             pass
16
17     return frames

```

Depois de definir todas as funções, o programa inicializa com as funções de inicialização do **pygame**, e define algumas variáveis importantes como **lista_dedos** e **área** que serão usadas como argumento para a função **FFT_functions**.

Em sequência, o programa carrega todas as imagens necessárias para a HUD do programa, seguindo a lógica:

```

1  sprite = pygame.image.load(
2      r"Simulations\Images\imagem.png"
3  ).convert_alpha()
4
5  sprite = pygame.transform.scale(sprite, (X, Y))

```

Que carrega a imagem em um formato fácil de ser manipulada e escala ela até o tamanho adequado.

Algo parecido é feito com as fontes, que são escritas em um formato de texto e depois formatadas e transformadas.

```

1  fonte_bem_pequena = pygame.font.Font(
2      r"Simulations\Images\Font\fonte", 1tamanho
3  )
4
5  str_texto = "..."
6

```



```

7     str_texto = justificar_texto(str_texto, largura da linha)
8     texto = fonte_bem_pequena.render(str_texto, True, (RGB da cor))

```

Com isso feito, o programa entra no loop principal do menu.

O loop consiste em 5 parte, uma para cada janela do menu. A janela é decidida por uma variável chamada **screen**, cada uma das janelas segue a mesma lógica: define o centro dos sprites, no caso de botões checa se o mouse está em cima para mudar o alpha (dando uma responsividade ao usuário), e desenha eles. A tela é redesenhada no final de cada ciclo do loop, com um fps fixado pelo **clock.tick(60)** a 60 fps.

Mesmo que elas sejam extremamente similares, algumas janelas são especiais por inicializar as outras simulações. Como dito em cima, o motor gráfico de um processo pode ser alterado ao abrir processos que o use em paralelo. Para evitar isso, o seguinte passo a passo é executado: o **pygame** é fechado para não interferir na performance das simulações, em seguida, as simulações são abertas como subprocessos, utilizando um motor gráfico próprio. Evitando, assim, que desconfigurem o principal.

```

1     pygame.quit()
2
3     base = os.path.dirname(__file__)
4
5     arquivo = os.path.join(base, "caminho simulacao")
6
7     subprocess.run(["python", arquivo])

```

O **base** pega o caminho até o arquivo atual e, a partir dele, é usado o caminho relativo para pegar o caminho até as outras simulações. Em **subprocess.run**, um argumento em lista é fornecido, ele diz para rodar o **arquivo** com o compilador python.

Essa estrutura resurge nas duas simulações visuais, mas para a **FFT_functions** uma lógica diferente é usada. Primeiro, o **pygame** não pode ser fechado, pois ele precisa mostrar os resultados. Por isso, é utilizada um script que gerá um gráfico não interativo, assim, o script principal não trava.

```

1     resultado, fragmento = fft.main(lista_dedos, area)
2
3     # Criando um arquivo para o subprocesso poder ler
4     dados = {
5         "resultado": resultado["reconstrucao"].tolist(),
6         "fragmento": fragmento.tolist(),
7     }
8
9     with open("dados.json", "w") as f:
10        json.dump(dados, f)

```

Contudo, isso gera um problema, o script que faz o gráfico não pode receber um input do menu, já que eles rodam em processos paralelos. Para isso, salva-se os resultados em um .json que recebe um dicionário com duas listas para poder gerar dois gráficos (a composição desses gráficos será explicada posteriormente na área do **FFT_functions**).

A função de reconstrução recebe duas entradas, a composição do sinal em dedos e a região de medição. Isso é decidido por uma interface gráfica com vários botões e um desenho ilustrativo de um braço.

Após essa etapa, o programa segue para expor os resultados. Há uma tabela escrita com as características do sinal reconstruído, expondo o erro, o score, as componentes previstas e as reais. Ademais, uma tela do **matplotlib** mostra o gráfico do sinal real e, em linhas tracejadas, o gráfico do sinal reconstruído.

Por fim, no final do loop há uma sessão dedicada a atualizar a tela e manter a frame-rate do menu. Logo após, há duas formas de sair do loop, uma usada quando abre-se uma simulação que visa reiniciar o menu, para evitar conflitos e erros, e outra para fechar a tela. As duas linhas finais é uma boa prática para evitar que o programa rode caso outra função o importe.

0.3 Código da simulação dos canais

Essa é a primeira simulação, ela usa duas interfaces gráficas e calcula suas operações dentro das operações do **numpy**.

```
1 import matplotlib
2
3 matplotlib.use("TkAgg")
4
5 import os # noqa: E402
6
7 os.environ["SDL_VIDEO_WINDOW_POS"] = "0,300"
8
9 import matplotlib.pyplot as plt # noqa: E402
10 import keyboard as kb # noqa: E402
11 import numpy as np # noqa: E402
12 import random # noqa: E402
13 import pygame # noqa: E402
14 # os comentarios sao so para nao aparecer um erro chato do ruff
```

Das bibliotecas importadas, somente a **random**, o **numpy** e o **matplotlib** não foram mencionados. A primeira é um biblioteca utilizada para gerar ou escolher elementos aleatórios de sequências, a segunda é uma biblioteca utilizada para realizar operações matemáticas mais avançadas, como multiplicação matricial, soluções lineares e distribuições estatísticas. Já a última, é uma biblioteca feita para criar gráficos, as duas linhas iniciais são utilizadas para trocar o compilador gráfico para um iterativo, capaz de criar uma imagem em tempo real.

Na parte das funções, elas não serão explicadas na ordem em que são definidas para uma melhor compreensão delas.

Para cada canal, teremos um vetor de estado de probabilidade, que define quantos por cento dos canais estão em cada estado. Para definir o estado inicial, o vetor deve estar em equilíbrio com o potencial inicial, caso contrário, há o risco dele ter uma falsa ativação no início. Para isso, a função **steady_state** acha o estado estacionário desse vetor. Todos esses estados, junto de outras variáveis de corrente, são colocados em um dicionário.

Matematicamente, a operação feita dentro do **steady_state** pode ser realizada pois o sistema

linear de A e b não são independentes, logo, eu posso trocar a última linha pela restrição de que $\sum_i \pi_i = 1$, sendo π o vetor probabilidade [5].

```

1  def steady_state(Q: np.array):
2      # Vamos definir o equilibrio inicial
3      A = Q.copy()
4      A[-1, :] = 1
5      b = np.zeros(Q.shape[0])
6      b[-1] = 1
7      return np.linalg.solve(A, b)

```

Essa e outras variáveis são definidas nesse início. Em sequência, o gráfico do **matplotlib** foi criado e suas componentes foram configuradas (legendas, variáveis, nomes, etc). Em sequência, o motor gráfico da biblioteca é posicionado em uma posição oposta a tela do **pygame**.

```

1  manager = plt.get_current_fig_manager()
2  manager.window.wm_geometry("+960+200")

```

As variáveis **V0**, **height** e **width** são definidas como globais para facilitar o seu uso dentro da classe e de outras funções.

O início da etapa do **pygame** é análoga ao menu, no sentido de configurar as janelas e carregar as imagens. Para desenhar a membrana, ela é segmentada em blocos com o mesmo tamanho dos sprites, quatro desses blocos são escolhidos ao acaso para tornar seu visual randômico. Com a posição da membrana definida, os íons foram desenhados.

```

1  class Ion:
2      def __init__(self, x, v, imagem, tipo, fator):
3          self.pos = np.array([x, v], dtype=complex)
4          self.imagem = imagem
5          # define qual animacao ela vai cumprir
6          self.tipo = tipo
7          # So para definir onde a imagem vai ser desenhada
8          self.rect = self.imagem.get_rect(
9              center=(np.real(self.pos[0]), np.imag(self.pos[0]))
10             )
11             self.fator = fator
12
13     # Movimento
14     def update(self, dt, ions, b_membrana, canais):
15         if self.tipo == "Fluxo Superior":
16             if np.real(self.pos[0]) > width:
17                 ions.remove(self)
18                 periodo = np.real(self.pos[0]) // 40
19                 self.pos[0] += (
20                     np.real(self.pos[1]) * dt
21                     + self.fator * np.sin(periodo + 5 * self.fator) *
22                     1j
23                 )

```

```

24     if self.tipo == "Fluxo Inferior":
25         if np.real(self.pos[0]) < 0:
26             ions.remove(self)
27         periodo = np.real(self.pos[0]) // 40
28         self.pos[0] += (
29             np.real(self.pos[1]) * dt
30             + self.fator * np.sin(periodo + 5 * self.fator) *
31                 1j
32         )
33     if (
34         self.tipo == "Na"
35         or self.tipo == "K"
36         or self.tipo == "L"
37         or self.tipo == "Ca"
38     ):
39         if np.imag(self.pos[0]) < 0 or np.imag(self.pos[0]) >
40             height:
41             ions.remove(self)
42         self.pos[0] += (
43             self.pos[1] * dt
44             + np.sin(np.imag(self.pos[0]) + np.real(self.pos
45                 [0])) / 5
46         )
47     if self.tipo == "Visual":
48         self.pos[0] += self.pos[1] * dt
49         self.rect.center = (np.real(self.pos[0]), np.imag(
50             self.pos[0]))
51
52         sprite_y = b_membrana.get_height()
53         sprite_x = b_membrana.get_width()
54         membr_y = height // 2
55
56         y, vy = np.imag(self.pos)
57         x, vx = np.real(self.pos)
58
59         np_x = float(x + vx * dt)
60         np_y = float(y + vy * dt)
61
62         # Na membrana
63         if abs(np_y - membr_y - sprite_y * (5 / 8)) <=
64             sprite_y / 4:
65             larg_canal = (9 / 25) * sprite_x
66             esp_canal = (8 / 25) * sprite_x
67             if esta_dentro_x(np_x, canais, larg_canal,
68                 esp_canal):

```

```

65         if abs(y - membr_y - sprite_y * (5 / 8)) <=
           sprite_y / 4:
66             self.pos[1] = -np.conj(self.pos[1])
67         else:
68             self.pos[1] = np.conj(self.pos[1])
69
70         # Condições que invertem o movimento nas fronteiras
71         if np_y <= 0 or np_y >= height:
72             self.pos[1] = np.conj(self.pos[1])
73
74         if np_x <= 0 or np_x >= width:
75             self.pos[1] = -np.conj(self.pos[1])
76
77         self.rect.center = (np.real(self.pos[0]), np.imag(self.
           pos[0]))
78
79     def draw(self, surface):
80         surface.blit(self.imagem, self.rect)

```

Dentro do objeto **Ions**, há 4 tipos de partículas: duas de fluxo contínuo, uma para cunho visual e uma para fluxo variável. As duas primeiras são fluxos senoidais que traduzem o potencial interno e externo, a única coisa que muda nelas durante a simulação é a sua densidade. O segundo tipo são partículas que não tem significado físico, somente fluem e colidem aleatoriamente para deixar a estética mais interessante. Por fim, as partículas de fluxo variável são aquelas que fluem nos canais, simbolizando a intensidade da corrente e a sua direção.

Toda essa mecânica foi configurada usando números complexos para facilitar operações de movimentação e colisão. A classe também tem outra função, sendo utilizada para redesenhar os íons depois de movimentarem.

Agora, a simulação entra no loop principal. Ela inicia definindo todos os comandos de teclado que serão utilizados, mas logo passa para operar na atualização do neurônio. Para diminuir o número de argumentos que entram na função e para atualizar todos os valores simultaneamente, cada uma das variáveis é comprimida em um vetor unidimensional. Ele entra como argumento na função **RK4**, a função derivada é dada por:

```

1     def f(t: float, y: np.array, k: dict, g: dict, C: float, Ir:
       float):
2         # Tau das sinapses
3         tau_E = 5.0
4         tau_I = 10.0
5
6         # Sigma dos noises dos canais
7         sigma_Na = 0.01 * 5
8         sigma_K = 0.006 * 5
9         sigma_Ca = 0.004 * 5
10
11         # Vetores estados de cada um
12         PNa = y[0:4]

```

```

13 PK = y[4:7]
14 PCa = y[7:10]
15 PL = y[10:13]
16 V = y[13]
17 Ca_i = y[14]
18 SynE = y[-2]
19 SynI = y[-1]
20
21 # Calculando as diferenciais no tempo
22 dPNa = Matriz_de_transicao("Na", k, V) @ PNa + noise_P(4,
    sigma_Na)
23 dPK = Matriz_de_transicao("K", k, V) @ PK + noise_P(3, sigma_K)
24 dPCa = Matriz_de_transicao("Ca", k, V0) @ PCa + noise_P(3,
    sigma_Ca)
25 dPL = Matriz_de_transicao("L", k, V) @ PL
26 dsE = dSyn_dt(SynE, tau_E)
27 dsI = dSyn_dt(SynI, tau_I)
28
29 # Eu vou colher as correntes tambem para o pygame
30 dV, IL, INa, IK, ICa = dV_dt(
31     t,
32     V,
33     {
34         "PNa": PNa,
35         "PK": PK,
36         "PCa": PCa,
37         "PL": PL,
38         "Ca_i": Ca_i,
39         "SynE": SynE,
40         "SynI": SynI,
41     },
42     g,
43     C,
44     Ir,
45 )
46
47 dCa = dCa_dt(-ICa, Ca_i)
48
49 # Produto de matriz
50 return (
51     np.concatenate([dPNa, dPK, dPCa, dPL, [dV], [dCa], [dsE], [
52         dsI]]),
53     IL,
54     INa,
55     IK,
56     ICa,
57 )

```

Para calcular a diferencial, usa-se duas funções: a matriz de transição e uma função noise. Essa segunda é uma função básica que gera valores aleatórios para simular o ruído biológico. Já a matriz de transição, ela é definida tendo em vistas as equações de equilíbrio químico dos canais:

```

1  def Matriz_de_transicao(nome: str, k: dict, V: float):
2      if nome != "Na":
3          K = k[nome][4](V)
4          # Cria a matriz de transicao
5          Q = np.array(
6              [[-K[0], K[1], 0], [K[0], -(K[1] + K[2]), K[3]], [0,
7                  K[2], -K[3]]],
8              dtype=float,
9          )
10         else:
11             K = k[nome][6](V)
12             # Cria a matriz de transicao
13             Q = np.array(
14                 [
15                     [-K[0], K[1], 0, K[5]],
16                     [K[0], -(K[1] + K[2]), K[3], 0],
17                     [0, K[2], -(K[3] + K[4]), 0],
18                     [0, 0, K[4], -K[5]],
19                 ],
20                 dtype=float,
21             )
22         return Q

```

Em que cada equilíbrio é explicado por uma função k_X , em que X é o alvo do equilíbrio. As equações das constantes foram ajustadas para obter um formato interessante no gráfico. Ao final da função f , é atualizado o potencial. As equações dessa função são resultados da manipulação do circuito.

```

1  def dV_dt(t: float, V: float, estados: dict, g: dict, C: float,
2      Ir: float):
3      # Simplesmente aplica a equacao diferencial do potencial
4      # Como ela e grande, vamos separar em componentes
5      comp1 = Ir / C # Corrente de estimulo
6      comp2 = -(g["L"][0] / C) * (V - g["L"][1]) # *estados["PL
7          "[2] #Corrente de vazao
8      comp3 = (
9          -(g["Na"][0] / C) * (V - g["Na"][1]) * estados["PNa"][2]
10         ) # Corrente de sodio
11     comp4 = (
12         -(g["K"][0] / C) * (V - g["K"][1]) * estados["PK"][2]
13     ) # Corrente de potassio
14     comp5 = (

```



```

13         -(g["Ca"][0] / C) * (V - g["Ca"][1]) * estados["PCa"][2]
14     ) # Corrente de calcio
15     comp6 = -(g["KCa"][0] / C) * (V - g["KCa"][1]) * KCa_open(
16         estados["Ca_i"])
17     # Corrente de acúmulo de calcio
18     # Corrente da sinapse Excitatoria
19     comp7 = -(g["SynE"][0] / C) * (V - g["SynE"][1]) * estados["
20         SynE"]
21     # Corrente da sinapse Inibitoria
22     comp8 = -(g["SynI"][0] / C) * (V - g["SynI"][1]) * estados["
23         SynI"]
24
25     return (
26         comp1 + comp2 + comp3 + comp4 + comp5 + comp6 + comp7 +
27         comp8 ,
28         comp2 ,
29         comp3 ,
30         comp4 ,
31         comp5 ,
32     )

```

Isso é tudo para a função de atualização. Ela é executada mais vezes para interpolar melhor os pontos intermediários. Ao terminar essa função, cada uma das variáveis são atualizadas e normalizadas para prevenir valores negativos de probabilidade.

Nesse estágio, já com as novas correntes e potenciais, uma certa quantidade de partículas é criada em cada corrente. Como variáveis tão sensíveis controlam o número de partículas, é fácil perceber uma variação dentro da janela visual.

O restante do código serve para atualizar o gráfico e configurar suas legendas. Ele tem uma limitação dinâmica na qual o gráfico se adapta ao tamanho da curva, possibilitando uma boa visualização em qualquer instante.

Ademais, todas as funções de atualização de imagem (que exigem um pequeno intervalo de tempo para serem processadas) ocorrem dentro de algumas iterações. Essa escolha foi feita na tentativa de acelerar a simulação ao mesmo tempo que se economiza performance sem impactar muito no visual.

Essa simulação foi criada com o intuito de ser uma representação gráfica das equações que modelam o neurônio, mas isso resulta na consequência de que ela é mais lenta para ser rodada. Na próxima simulação, há uma inversão dessa necessidade.

0.4 Código da simulação de um EEG

Em geral, o código da simulação do EEG é extremamente parecido com a anterior, sendo o principal fator destoante a falta do visualizador **pygame**, uma rede organizada em grafos e a lógica de conexão.

Pela falta de tempo, uma janela visual da rede de neurônios não foi criada. Além de já existir duas janelas com informações, tornando uma terceira uma adição poluente, o programa tem um intervalo de iterações que tornariam a visualização pouco interessante.

Contudo, uma janela adicional foi criada. O primeiro gráfico sobrepõem todos os potenciais de membrana dos neurônios. Por mais que seja útil para controlar como a simulação está avançando e a taxa de reativação dos componentes, ela contém uma quantidade excessiva de informações. Por isso, uma segunda tela foi gerada.

```

1  for n in neuronios:
2      # Atualizando a lista dentro dos neuronios
3      n.lista_V = atualizacao(n.lista_V, n.estado["V"])
4      n.estado["V_pico"] = -65
5      V = n.estado["V"]
6      EEG += g["SynE"][0] * n.estado["SynE"] * (V - g["SynE"][1]) +
7            g["SynI"][
8                ] * n.estado["SynI"] * (V - g["SynI"][1])
9
10     V_soma = gain * EEG
11
12     # Perda com o tempo
13     EEG *= 0.1
14
15     lista_VT = atualizacao(lista_VT, V_soma, "Total")
16     if gravar:
17         sinal = atualizacao(sinal, float(V_soma), "Total")
18
19     if frame % rende_todo == 0:
20         for n in neuronios:
21             # Atualizando a lista dentro dos neuronios
22             n.line.set_data(lista_t, n.lista_V)
23
24             # Para o grafico total
25             lineT.set_data(lista_tT, lista_VT)
26             minimo = lista_tT[-500] if len(lista_tT) > 500 else min(
27                 lista_tT)
28             ax_total.set_xlim(minimo, lista_tT[-1])

```

Ela se preocupa menos com o comportamento individual e mais com a soma do coletivo. Um EEG detecta as correntes sinápticas, por isso, os valores da corrente de sinapse excitatória e inibitória são somados e adicionados em uma lista que, posteriormente, gerará o gráfico do EEG. Ele permite analisar o comportamento de spike dos neurônios sem nenhum outro fator interferindo na análise.

Contudo, para realizar essa conectividade, os neurônios foram transformados em classes de objetos, armazenando suas próprias variáveis, e organizados em um grafo.

```

1  class Neuronio:
2      """Objeto do neuronio para mexer nele"""
3
4      def __init__(
5          self,

```

```

6         estado: dict,
7         k: dict,
8         lista_V: list,
9         line: plt.figure,
10        ax: plt.axes,
11        lista_t: list,
12        number: float,
13        V0=-65,
14    ):
15
16        self.k = {"Na": k_Na, "K": k_K, "Ca": k_Ca, "L": k_L}
17        self.estado = {
18            "PNa": steady_state(Matriz_de_transicao("Na", self.k,
19                V0)),
19            "PK": steady_state(Matriz_de_transicao("K", self.k,
20                V0)),
20            "PCa": steady_state(Matriz_de_transicao("Ca", self.k,
21                V0)),
21            "PL": np.array([1, 0, 0], dtype=float),
22            "V": V0,
23            "Ca_i": 0.0001,
24            "SynE": 0.0,
25            "SynI": 0.0,
26            "V_anterior": V0,
27            "V_ant_anterior": 0,
28            "spike": False,
29            "V_pico": V0,
30        }
31        self.lista_V = atualizacao([], self.estado["V"])
32        (self.line,) = ax.plot(lista_t, self.lista_V, label=f"V{
33            number + 1}(t)")
34        self.number = number

```

```

1    conexoes = [
2        # Anel excitatorio principal
3        {"pre": 0, "pos": 1, "tipo": "E", "peso": 0.65},
4        {"pre": 1, "pos": 2, "tipo": "E", "peso": 0.60},
5        {"pre": 2, "pos": 3, "tipo": "E", "peso": 0.65},
6        .
7        .
8        .
9
10       # Inibitorias
11       {"pre": 5, "pos": 10, "tipo": "I", "peso": 0.35},
12       {"pre": 10, "pos": 15, "tipo": "I", "peso": 0.35},
13       .
14       .

```

```

15         .
16
17         # Conexoes excitatorias aleatorias fracas
18         {"pre": 0, "pos": 10, "tipo": "E", "peso": 0.15},
19         .
20         .
21         .
22
23     ]

```

Esse grafo identifica qual o neurônio disparador e qual o receptor, qual o tipo de ligação e a força dela. Isso permite iterar em cada conexão e, ao detectar que um spike ocorreu, dar um estímulo correto ao receptor.

Uma última diferença entra ambas simulações está na aplicação da função noise dentro dos neurônios. Para fins de performance, ela não é mais calculada e aplicada nas quatro componentes do RK4, se retendo ao último componente. Isso acelera o cálculo, mas atenua um pouco o efeito da função, já que dentro do passo ele é ponderado, diminuindo sua intensidade por 6.

0.5 Código decompositor de sinal por Fourier

A lógica e objetivo dessa simulação é radicalmente diferente das anteriores. Enquanto as outras simulações recebiam inputs e geravam um resultado, essa realiza a operação oposta. Antes de explicar o seu mecanismo interno, segue algumas funções auxiliares para a sua operação.

O programa precisa ter, dentro de sua memória, sinais de referência para tentar decompor. Por isso, é necessário gerar os sinais e salva-los em uma biblioteca consultável. Para isso que o arquivo **bib_eeg.pkl** serve, ele é uma biblioteca capaz de salvar array's do **numpy**. Nele, um dicionário de sinais é compilado e posteriormente carregado.

```

1  def salvar_assinatura(nome: str, sinal: list, tempo: list):
2      """Salva a assinatura do sinal em um array"""
3      # Converte no jeito de gente, uso o copy para nao modificar a
4          lista original
5      signal = np.asarray(sinal.copy())
6      time = np.asarray(tempo.copy())
7
8      # Acha a frequencia do sinal, o diff calcula a diferenca
9          entre
10     # Amostras consecutivas
11     dt_ms = np.mean(np.diff(time))
12     fs = 1000 / dt_ms
13
14     freqs, coef, media = Decomposer_FFT(signal, fs)
15
16     biblioteca = carrega_bib()
17
18     biblioteca[nome] = {
19         "fs": fs,

```

```

18         "n": len(signal),
19         "media": media,
20         "freqs": freqs,
21         "coef_real": coef.real,
22         "coef_imag": coef.imag,
23     }
24
25     salva_bib(biblioteca)
  
```

Para salvar um sinal, existe a função que recebe o nome, o sinal e o a marcação de tempo para decompor o sinal em parcelas salváveis. Ele começa transformando o sinal e o tempo em um array, que serão direcionados aos seus devidos tratamentos.

```

1 def salvar_assinatura(nome: str, sinal: list, tempo: list):
2     """Salva a assinatura do sinal em um array"""
3     signal = np.asarray(sinal.copy())
4     time = np.asarray(tempo.copy())
5
6     dt_ms = np.mean(np.diff(time))
7     fs = 1000 / dt_ms
8
9     freqs, coef, media = Decomposer_FFT(signal, fs)
10
11     biblioteca = carrega_bib()
12
13     biblioteca[nome] = {
14         "fs": fs,
15         "n": len(signal),
16         "media": media,
17         "freqs": freqs,
18         "coef_real": coef.real,
19         "coef_imag": coef.imag,
20     }
21
22     salva_bib(biblioteca)
  
```

O tempo colocado dentro de uma **mean(diff())**, que calcula a diferença média entre os termos, usada para calcular a frequência dos pontos. Isso e o sinal são alimentados em uma função que decompõe-o em coeficientes e uma frequência da onda.

```

1 def Decomposer_FFT(sinal: np.ndarray, fs: float):
2     """Usando FFT, decompoe o sinal em varias componentes, util para
3     guardar e fazer
4     operacoes futuras"""
5
6     # Garante que ta no formato certo, em teoria nao muda nada alem
7     do formato
8     sinal = np.asarray(sinal, dtype=float)
  
```

```

8      # remove componente Media, para o fft so ver as oscilacoes sem
9      # deslocamento vertical
10     media = np.mean(sinal)
11     x = sinal - media
12
13     # FFT
14     coef = rfft(x)
15     freqs = rfftfreq(len(x), d=1 / fs)
16
17     return freqs, coef, media

```

Isso é o que é denominado de assinatura do sinal, é uma forma mais simples de salva-lo e, caso necessite, basta o reconstruir. Tudo isso é salvo e com as funções que buscam no arquivo da biblioteca de sinais.

```

1 def carrega_bib():
2     """Garante que eu receba algo ao tentar carregar a minha
3     biblioteca"""
4     if not bib_path.exists() or bib_path.stat().st_size == 0:
5         return {}
6
7     with open(bib_path, "rb") as f:
8         return pickle.load(f)
9
10 def salva_bib(biblioteca: dict):
11     """Garante que eu salve a minha biblioteca no arquivo bib_eeg"""
12     bib_path.parent.mkdir(parents=True, exist_ok=True)
13
14     with open(bib_path, "wb") as f:
15         pickle.dump(biblioteca, f)

```

Quando o script recebe uma lista, seu primeiro passo é carregar todos os sinais dentro da biblioteca. Isso envolve o processo inverso apresentado pelo **Decompouser_FFT**. A lista codifica quais sinais participam do fragmento principal, usando uma função denominada **criar_frag_teste**, essa lista soma os sinais e corta eles no intervalo de análise.

Com esse fragmento, entra-se no corpo principal do script. O fragmento é normalizado para não haver problemas com amplitude (um foco maior na forma do sinal), o programa é capaz de processar até 10 sinais como componentes, acima disso, ele considera que a maioria da composição dos outros sinais é desprezível.

Em sequência, os melhores k candidatos são varridos com um passo p. A função **top_candidatos_finder** organiza os resultados e filtra os top k candidatos. A função **montar_dicionario_janelas** combina os candidatos da próxima iteração em um dicionário para ser testado pelo OMP, esse dicionário contém todas as combinações a serem testadas.

```

1     modelo = OrthogonalMatchingPursuit(n_nonzero_coefs=n_comp,
2         fit_intercept=False)

```



```

3      # Um objeto que acha o coeficiente que melhor explica o residuo
4      modelo.fit(D, y)
5
6      coef = modelo.coef_
7      # Reconstrucao do sinal original
8      recon = D @ coef
9
10     erro = np.linalg.norm(y - recon) / (np.linalg.norm(y) + 1e-12)
11     score = float(np.dot(y, normaliza_sinal(recon)))
12
13     # Retorna o indice de todos os elementos maiores que e^-12
14     idx = np.flatnonzero(np.abs(coef) > 1e-12)
15
16     componentes = []
17
18     for i in idx:
19         componentes.append(
20             {
21                 "nome": metadados[i]["nome"],
22                 "inicio": metadados[i]["inicio"],
23                 "fim": metadados[i]["fim"],
24                 "peso": float(coef[i]),
25                 "score_janela": metadados[i]["score"],
26             }
27         )

```

Em sequencia, todos os candidatos com coeficiente não zero são adicionados nas componentes, que, posteriormente, é organizada e rankeada para passar novamente no processo de filtragem e recomposição. Na última iteração, as componentes são rankeadas entre si, usando uma multiplicação matricial para definir qual combinação de componenetes resulta no melhor score.

Por último, isso é compilado em um dicionário de resultados, que apresenta a melhor composição mas também contém o score individual dos candidatos (sendo possível, em um futuro, usar isso para avaliar outras composições).

0.6 Código plot do fragmento

Essa sessão será muito curta. O script **plot_frag** só é usado para ler o arquivo .json feito pelo menu e plotar um gráfico. Seu código completo é exposto a seguir.

```

1      import matplotlib
2
3      matplotlib.use("TkAgg")
4
5      import matplotlib.pyplot as plt
6      import numpy as np
7      import json

```

```

8
9
10 with open("dados.json") as f:
11     dados = json.load(f)
12
13
14     fragmento = np.array(dados["fragmento"])
15     resultado = np.array(dados["resultado"])
16
17     # Plotando resultado para se bonitinho :3
18     plt.plot(fragmento, label="Sinal original")
19     plt.plot(
20         resultado,
21         linestyle="--",
22         label="Reconstrucao",
23     )
24     plt.title("fragmento artificial (feche essa janela para voltar ao
25         menu)")
26     plt.legend()
27     plt.grid()
28     plt.show()

```

Conclusões

O programa como um todo simula de uma forma consistente neurônios isolados e em rede, além de ter uma consistência interessante na previsão de composição de sinais diferenciáveis. A ideia foi concluída, contendo todas as partes planejadas para essa simulação. Por fim, o programa representou um grande desafio ao aluno, já que envolveu a pesquisa de conceitos desconhecidos e de bibliotecas novas.

Referências

- 1 GERSTNER, W. et al. *The Hodgkin-Huxley Model*. 2014. Chapter 2.2 of the online textbook Neuronal Dynamics. Disponível em: <<https://neurondynamics.epfl.ch/online/Ch2.S2.html>>.
- 2 COLQUHOUN, D.; HAWKES, A. G. The principles of the stochastic interpretation of ion-channel mechanisms. In: SAKMANN, B.; NEHER, E. (Ed.). *Single-Channel Recording*. 2. ed. New York: Plenum Press, 1995. p. 397–482.
- 3 CHAPRA, S. C.; CANALE, R. P. *Fourth-Order Runge-Kutta Method*. McGraw-Hill, 2010. 727–735 p. Disponível em: <https://www.mbit.edu.in/wp-content/uploads/2020/05/Numerical_methods_for_engineers_for_engi.pdf>.

- 4 TROPP, J. A.; GILBERT, A. C. Signal recovery from random measurements via orthogonal matching pursuit: The gaussian case. *Preprint*, 2007. Available online. Disponível em: <<https://tropp.caltech.edu/papers/TG07-Signal-Recovery-preprint.pdf>>.
- 5 BOLDRINI, J. L. et al. *Álgebra Linear*. 3. ed. São Paulo: Harper & Row do Brasil, 1980.
- 6 Modelo de Linguagem de Grande Porte (LLM). *Uso de modelo de linguagem de grande porte como ferramenta de apoio à pesquisa e revisão textual*. 2026. Modelo de linguagem de grande porte (LLM) utilizado como ferramenta de apoio para pesquisa conceitual e correção ortográfica e gramatical do texto, sem substituição da elaboração autoral do conteúdo científico.